**Выполнил: Василенко И.А Группа: ПИа-090303-о24**

# ЛАБОРАТОРНАЯ РАБОТА №21

## Дисциплина: Основы программирования на Python

## Тема: Итоговый проект: разработка, тестирование и развёртывание прикладной Python-системы

## Вариант 2: Система управления библиотекой

### Цель работы

Закрепить и систематизировать знания, полученные в ходе выполнения лабораторных работ курса. В рамках итогового проекта необходимо самостоятельно пройти основные этапы разработки программной системы: анализ требований, проектирование, разработку, работу с данными, реализацию API, тестирование, контейнеризацию и развёртывание, а также подготовить документацию и оценить ограничения полученного решения.

### Постановка индивидуального задания (Вариант 2. Система управления библиотекой)

Разработать прикладную систему управления библиотекой с сущностями Reader (читатель), Book (книга), Author (автор) и Loan (выдача). Реализовать выдачу и возврат книг, каталог с поиском, фильтрацией и сортировкой, справочники авторов и читателей, разграничение прав доступа и статистику. Система должна содержать базу данных, REST API, механизм авторизации, набор автоматических тестов, конфигурацию Docker Compose и конвейер непрерывной интеграции.

### Описание предметной области и функциональные требования

Библиотека ведёт учёт книжного фонда и выдач. Читатель регистрируется в системе, получает доступ к каталогу и может взять книгу на установленный

срок, а затем вернуть её. Сотрудник библиотеки управляет фондом: добавляет и изменяет книги и авторов, выдаёт и принимает книги, контролирует задолженности.

1. Регистрация и вход читателя с выдачей токена доступа.
2. Просмотр и поиск книг по названию, автору и году, фильтрация по наличию, сортировка и пагинация.
3. Просмотр справочника авторов и карточек читателей.
4. Выдача книги читателю с соблюдением бизнес-правил.
5. Возврат книги с восстановлением числа доступных экземпляров.
6. Просмотр выдач с фильтрацией по читателю, книге и признаку возврата.
7. Управление книгами и авторами для пользователей с ролью администратора.
8. Проверка работоспособности системы по служебному маршруту.

## Нефункциональные требования

| Требование | Реализация |
|---|---|
| Производительность | Индексы по внешним ключам и полям поиска, пагинация выдачи, отсутствие запросов в цикле |
| Безопасность | Хеширование паролей с солью, токены доступа с ограниченным сроком, разграничение ролей, отсутствие трассировки в ответах |
| Надёжность | Проверки бизнес-правил на уровне сервиса, ограничения целостности на уровне базы, централизованная обработка ошибок |
| Сопровождаемость | Разделение на слои, аннотации типов, единый стиль, линтер Ruff в конвейере |
| Развёртывание | Docker-образ приложения, PostgreSQL в отдельном контейнере, переменные окружения |
| Наблюдаемость | Журналирование запросов с кодом ответа и длительностью, служебный маршрут /health |

## Архитектура системы

Система построена по слоистой схеме. Клиент обращается к REST API, маршруты вызывают сервисы, сервисы работают с репозиториями, а

репозитории — с базой данных через SQLAlchemy. Такое разделение позволяет тестировать бизнес-логику без HTTP и менять хранилище без правки маршрутов.

| Слой | Компоненты и ответственность |
|---|---|
| API | api/books.py, api/authors.py, api/readers.py, api/loans.py, api/auth_routes.py — приём запросов, схемы, коды состояния |
| Зависимости | api/deps.py — создание сервисов и проверка токена через Depends |
| Сервисы | services/auth.py, books.py, authors.py, loans.py — бизнес-правила и проверки |
| Репозитории | repositories/base.py и наследники — типовые операции и специализированные запросы |
| Модели и схемы | models.py — ORM-модели, schemas.py — схемы Pydantic |
| Инфраструктура | database.py, config.py, exceptions.py, utils/logging.py — подключение, настройки, ошибки, журналирование |
| Точка входа | main.py — создание приложения, обработчики ошибок, middleware, маршрут /health |

## Структура базы данных

| Таблица | Поля | Связи и ограничения |
|---|---|---|
| readers | id, username, email, hashed_password, role, is_active, created_at | Уникальные username и email, роль USER или ADMIN |
| authors | id, name, country, birth_year | Один автор — много книг |
| books | id, title, author_id, year, isbn, genre, total_copies, available_copies | FK на автора, уникальный ISBN, ограничение available <= total |
| loans | id, book_id, reader_id, issue_date, due_date, return_date, returned | FK на книгу и читателя, ограничение due >= issue |

Связь книги с автором задана внешним ключом, а выдачи связывают книгу и читателя и хранят даты выдачи, срока и возврата. Поле returned дублирует факт возврата для быстрой фильтрации, а число доступных экземпляров уменьшается при выдаче и увеличивается при возврате, что позволяет быстро определить наличие книги без подсчёта выдач.

## REST API системы

| Метод и путь | Доступ | Назначение |
|---|---|---|
| POST /auth/register | без токена | Регистрация читателя, возврат токена |
| POST /auth/login | без токена | Вход и получение токена |
| GET /books | любой авторизованный | Каталог с поиском, фильтром и сортировкой |

| Метод и путь | Доступ | Назначение |
|---|---|---|
| GET /books/{id} | любой авторизованный | Карточка книги с автором |
| POST /books | ADMIN | Создание книги |
| PUT /books/{id}, DELETE /books/{id} | ADMIN | Изменение и удаление книги |
| GET /authors, POST /authors | чтение — все, создание — ADMIN | Справочник авторов |
| GET /readers/me | любой авторизованный | Профиль текущего читателя |
| GET /readers | ADMIN | Список читателей |
| POST /loans | любой авторизованный | Выдача книги |
| POST /loans/{id}/return | владелец выдачи или ADMIN | Возврат книги |
| GET /loans | любой авторизованный | Список выдач с фильтрацией |
| GET /health | без токена | Проверка работоспособности |

Все ответы описаны схемами Pydantic, а списки возвращаются объектом Page с полями items, total и параметрами страницы. Ошибки формируются единообразно: поле error содержит машинный код, поле message — понятное описание, что позволяет клиенту различать ситуации без разбора текста.

### Бизнес-логика выдачи и возврата

Правила предметной области сосредоточены в сервисе выдач, что исключает их дублирование в маршрутах. Проверяются состояние читателя, наличие свободных экземпляров, отсутствие просроченных выдач, непревышение лимита одновременных выдач, активность выдачи при возврате и корректность дат.

1. Нельзя выдать книгу, если у неё нет свободных экземпляров.
2. Нельзя выдать книгу читателю, у которого есть просроченные выдачи.
3. Нельзя превысить максимальное число одновременных выдач, заданное настройкой.
4. Возвратить можно только активную, ещё не возвращённую выдачу.
5. Дата возврата не может быть раньше даты выдачи.

6. Нельзя выдать несуществующую книгу или книгу деактивированному читателю.

Листинг 21.1 — Правила выдачи книги (services/loans.py)

```
def issue(self, reader: Reader, data: LoanCreate) -> Loan:
    book = self.books.get(data.book_id)
    if not reader.is_active:
        raise BusinessRuleException(
            "Читатель деактивирован, выдача невозможна")


    if book.available_copies <= 0:
        raise BusinessRuleException(
            f"Книга {book.title!r} временно недоступна: "
            f"нет свободных экземпляров")


    today = data.issue_date or date.today()
    if self.loans.has_overdue(reader.id, today):
        raise BusinessRuleException(
            "У читателя есть просроченные выдачи; сначала верните их")


    active = self.loans.count_active_for_reader(reader.id)
    if active >= settings.max_active_loans:
        raise BusinessRuleException(
            f"Читатель достиг лимита одновременных выдач "
            f"({settings.max_active_loans})")


    loan_days = data.loan_days or settings.loan_days
    due = today + timedelta(days=loan_days)


    loan = Loan(book_id=book.id, reader_id=reader.id, issue_date=today,
                due_date=due, returned=False)
    book.available_copies -= 1
    self.loans.add(loan)
    return loan
```

**Авторизация и разграничение доступа**

Пароли не хранятся в открытом виде: функция hash_password() формирует случайную соль и вычисляет дайджест, а verify_password() сравнивает результат повторного вычисления, используя сравнение за постоянное время. Вход возвращает токен доступа, в котором закодированы идентификатор пользователя и его роль, а также срок действия.

Зависимость get_current_reader() извлекает токен из заголовка Authorization, проверяет подпись и срок, после чего загружает читателя из базы. Функция require_role() возвращает зависимость, которая отклоняет запрос с кодом 403, если роль пользователя не входит в разрешённый набор. Возврат

книги дополнительно проверяет, что выдача принадлежит текущему пользователю, если он не администратор.

Листинг 21.2 — Проверка токена и ролей (auth.py)

```
def create_access_token(reader_id: int, role: str) -> str:
    payload = {"sub": str(reader_id), "role": role,
               "exp": datetime.now(timezone.utc)
                      + timedelta(minutes=settings.token_ttl_minutes)}
    return jwt.encode(payload, settings.secret_key, algorithm="HS256")


def get_current_reader(token: str = Depends(oauth2_scheme),
                       db: Session = Depends(get_session)) -> Reader:
    try:
        payload = decode_token(token)
    except JWTError:
        raise AuthenticationException("Некорректный или истёкший токен")
    reader = ReaderRepository(db).get_or_none(int(payload["sub"]))
    if reader is None or not reader.is_active:
        raise AuthenticationException("Пользователь не найден")
    return reader


def require_role(*roles: str):
    def _dep(reader: Reader = Depends(get_current_reader)) -> Reader:
        if reader.role not in roles:
            raise AuthorizationException("Недостаточно прав")
        return reader
    return _dep
```

**Централизованная обработка ошибок**

Иерархия доменных исключений наследуется от DomainError и содержит код, сообщение и код состояния HTTP: EntityNotFoundException даёт 404, ValidationException — 400, AuthenticationException — 401, AuthorizationException — 403, ConflictException — 409, BusinessRuleException — 422, ExternalServiceException — 502. Обработчик в main.py преобразует любое доменное исключение в ответ с полями error и message, ошибки валидации Pydantic возвращаются с кодом 422, а непредвиденные исключения журналируются полностью, но клиенту отдаётся общее сообщение без трассировки. Такой подход исключает раскрытие внутренних деталей реализации.

## Тестирование

Тесты разделены по уровням и используют подмену зависимости базы данных на временную SQLite, создаваемую для каждого теста. Всего выполнено шестьдесят два теста.

| Файл тестов | Кол-во | Что проверяется |
|---|---|---|
| tests/test_auth_unit.py | 14 | Формат хеша, проверка верного и неверного пароля, случайность соли, устойчивость к повреждённому хешу, содержимое и проверка токена, отказ при подделке |
| tests/test_auth_api.py | 16 | Регистрация и выдача токена, дубликаты логина и почты, валидация полей, вход с неверным паролем, отсутствие и порча токена, доступ к профилю, запрет операций для обычного пользователя, ответ 404 |
| tests/test_books_api.py | 6 | Полный цикл CRUD книги, ответ 404, отсутствующий автор, поиск, фильтрация и сортировка, ошибка валидации, запрет удаления выданной книги |
| tests/test_loans_api.py | 6 | Выдача и возврат, отказ при отсутствии экземпляров, отсутствующая книга, превышение лимита, запрет возврата чужой выдачи, фильтрация по признаку возврата |
| tests/test_services.py | 17 | Отсутствующая книга и автор, равенство доступных и общих экземпляров, выдача и возврат, отказ при недоступной книге, лимит выдач, просрочка, повторный возврат, некорректная дата, деактивированный читатель, согласованность при уменьшении тиража |
| tests/test_mocks.py | 3 | Проверка вызовов репозиториев сервисом выдач с помощью Mock |

Все тесты проходят успешно. Конвейер дополнительно контролирует покрытие кода: команда pytest --cov=src --cov-report=term-missing --cov-fail-under=70 завершает сборку ошибкой при падении покрытия ниже установленного порога.

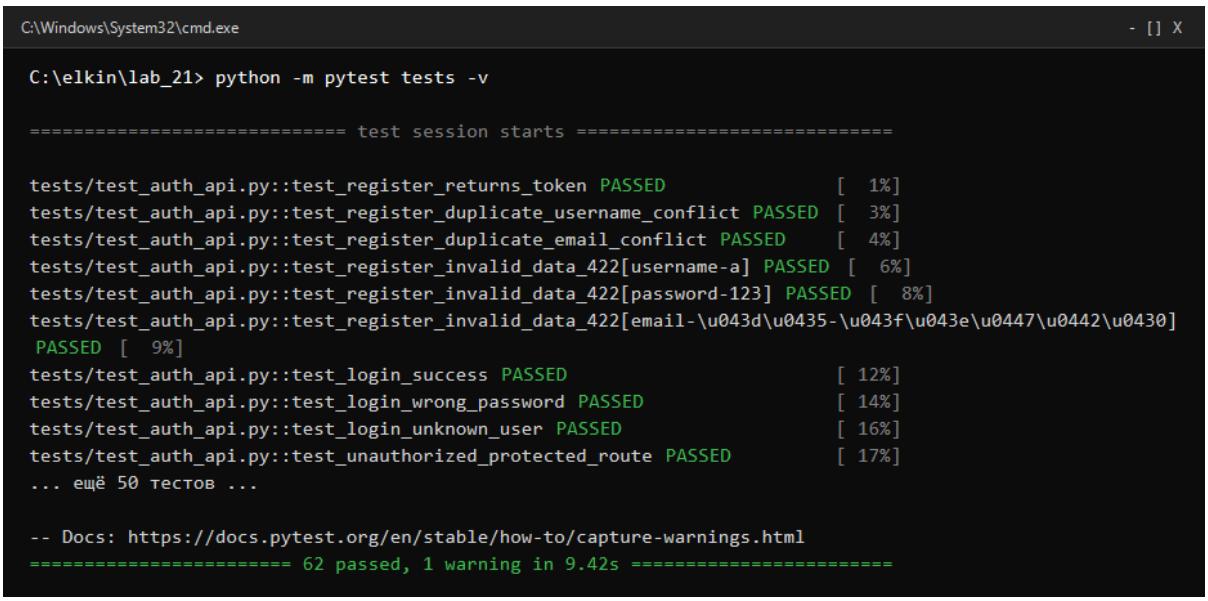



Рисунок 21.1 — Результат прохождения тестов итогового проекта

**Контейнеризация и развёртывание**

Приложение собирается в образ на основе python:3.12-slim. Dockerfile устанавливает зависимости, запускает приложение от непривилегированного пользователя и передаёт управление gunicorn с рабочими процессами uvicorn. Docker Compose описывает два контейнера: приложение и PostgreSQL с томом для данных, общей сетью и проверкой готовности базы командой pg_isready; приложение запускается только после перехода базы в состояние healthy. Параметры подключения, уровень логирования и секретный ключ передаются через переменные окружения, описанные в файле .env.example, а сам файл .env исключён из репозитория.

Листинг 21.3 — Фрагмент docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${DATABASE_NAME:-library}
      POSTGRES_USER: ${DATABASE_USER:-library}
      POSTGRES_PASSWORD: ${DATABASE_PASSWORD:-library_password}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U library -d library"]
      interval: 10s
      retries: 5

  app:
    build: .
    depends_on:
```

```
    db:
      condition: service_healthy
  environment:
    DATABASE_HOST: db
    USE_SQLITE: "false"
  ports:
    - "8000:8000"
```

**Конвейер непрерывной интеграции**

Файл .github/workflows/ci.yml описывает три задачи. Задача lint выполняет проверку стиля линтером Ruff, задача test запускает pytest с измерением покрытия и порогом в семьдесят процентов, задача build собирает Docker-образ и выполняется только при успехе предыдущих. Кэширование зависимостей pip и слоёв образа сокращает время повторных прогонов. Такой конвейер обеспечивает, что в основную ветку попадает только код, прошедший статические проверки, тесты и сборку.

**Анализ проблем и ограничений**

В ходе разработки выявлены следующие ограничения решения. Аутентификация построена на собственной реализации токенов без механизма обновления, поэтому по истечении срока действия требуется повторный вход. Роли сведены к двум значениям, что достаточно для учебной системы, но не покрывает сценарии с библиотекарями разных уровней. Отсутствует журналирование изменений предметных объектов в отдельном журнале аудита, а уведомления о просроченных выдачах формируются только по запросу, без фоновой рассылки. Тесты выполняются на SQLite, поэтому отдельные особенности PostgreSQL, например поведение типов дат, проверены только при запуске в контейнере.

Перспективы развития: добавить обновление токенов и подтверждение электронной почты, реализовать фоновую задачу рассылки напоминаний о просрочке, ввести журнал аудита операций, расширить роли и права, добавить кэширование каталога и полнотекстовый поиск, а также перейти к нагрузочному тестированию и мониторингу с измерением времени ответа.

## Листинги ключевых фрагментов программы

Листинг 21.4 — Создание приложения и обработка ошибок (main.py)

```
def create_app() -> FastAPI:
    app = FastAPI(title=settings.app_name, version="1.0.0",
                  docs_url="/swagger",
                  openapi_url="/swagger/openapi.json")
    init_db()


    @app.exception_handler(DomainError)
    async def domain_error_handler(request: Request, exc: DomainError):
        resp = ErrorResponse(exc.code, exc.message, exc.status_code)
        logger.warning("%s %s -> %s %s", request.method,
                       request.url.path, exc.code, exc.message)
        return JSONResponse(status_code=exc.status_code,
                            content=resp.to_dict())


    @app.exception_handler(Exception)
    async def unhandled_handler(request: Request, exc: Exception):
        logger.exception("Необработанная ошибка %s %s",
                         request.method, request.url.path)
        return JSONResponse(status_code=500,
                            content={"error": "INTERNAL_SERVER_ERROR",
                                     "message": "Внутренняя ошибка сервера"})


    app.include_router(api_router)
    return app
```

Листинг 21.5 — Базовый репозиторий (repositories/base.py)

```
class BaseRepository(Generic[ModelT]):
    def __init__(self, db: Session):
        self.db = db


    def get(self, entity_id: int) -> ModelT:
        instance = self.get_or_none(entity_id)
        if instance is None:
            raise EntityNotFoundException("Объект не найден")
        return instance


    def get_or_none(self, entity_id: int) -> ModelT | None:
        return self.db.get(self.model, entity_id)


    def list_all(self) -> list[ModelT]:
        return list(self.db.scalars(select(self.model)).all())


    def add(self, instance: ModelT) -> ModelT:
        self.db.add(instance)
        self.db.flush()
        return instance
```

## Результаты демонстрации

Работа системы проверялась в демонстрационном режиме и через интерактивную документацию /swagger. Проверены регистрация и вход, получение профиля, просмотр каталога с поиском и сортировкой, создание книги

под учётной записью администратора, выдача книги читателю, попытка выдачи при отсутствии экземпляров, возврат книги, просмотр выдач с фильтрацией и отказ в доступе к административным операциям для обычного пользователя.

| **Действие** | **Ожидаемый и полученный результат** |
|---|---|
| Регистрация и вход | Токен получен, профиль доступен по /readers/me |
| Создание книги без прав | Ответ 403 с кодом AUTHORIZATION_ERROR |
| Создание книги администратором | Ответ 201, книга появляется в каталоге |
| Выдача книги | Ответ 201, число доступных экземпляров уменьшилось |
| Повторная выдача без экземпляров | Ответ 422 с сообщением о недоступности |
| Возврат книги | Ответ 200, экземпляр снова доступен |
| Запрос с истёкшим токеном | Ответ 401 с кодом AUTHENTICATION_ERROR |
| Проверка /health | Ответ со статусом healthy |

## Вывод

В ходе выполнения итогового проекта разработана система управления библиотекой, прошедшая все основные этапы создания программного продукта: анализ требований, проектирование базы данных, реализацию слоистой архитектуры, разработку REST API, обеспечение безопасности, тестирование, контейнеризацию и настройку непрерывной интеграции. Система поддерживает регистрацию и авторизацию с разграничением ролей, полный набор операций над книгами и авторами, выдачу и возврат книг с соблюдением шести бизнес-правил, фильтрацию и пагинацию. Шестьдесят два автоматических теста покрывают модули авторизации, сервисы и маршруты, конвейер контролирует стиль, покрытие и сборку образа, а Docker Compose обеспечивает воспроизводимый запуск приложения вместе с базой данных PostgreSQL.